ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Semester Project Proposal

NuActionGUI

Supervisor: Dr. Srđan Krstić, Hoang Nguyen
Professor: Prof. David Basin

1 Prerequisites

Familiarity with databases, object-relational mappers, OCL, and model-driven development.[1]

2 Motivation

Enforcing security (e.g., fine-grained access control [8]) and privacy (e.g., purpose limitation [5]) policies in today's data-centric applications is both critical and challenging. It is critical because of the high economic and social consequences of enforcement failure. It is challenging as it is not yet well-supported by the current technologies: it is implemented in an ad-hoc manner, which makes its maintenance and evolution error-prone and time consuming. In particular, developers must address the policies in a consistent and cross-cutting manner. As a result, broken access control is the most common web application vulnerability according to the OWASP Top 10 report [7], whereas data protection violation fines have reached an all time high [4].

Previously, Basin et al. [3] have proposed a model-driven security (MDS) approach for building secure-by-design systems. In addition to the application, they model security policies expressed as fine-grained access control (FGAC) permissions. The idea is that software engineer first creates a design model in a design modeling language (e.g., a UML class diagram), which represents the system's state-space. A security engineer then creates a security model in a security modeling language, which defines the system's FGAC permissions. These permissions consist of the (static) role-based access control specification and the (dynamic) OCL constraints. Using model transformations, a enforcement code is generated from the two models. ActionGUI [2] is a tool that supports the above MDS methodology and generates fully functional web applications with a configured enforcement mechanism for the FGAC policies specified by the security model. To generate fully functional applications, ActionGUI additionally requires a third (GUI) model that specifies the web application's page layout and behavior. While ActionGUI was used for building real applications and in an academic setting, its potential never materialized into wide-spread use. This is due to a couple of drawback which makes it difficult to use. Firstly, it does not provide generated code that can be easily adjusted, which means that it cannot be used with other useful libraries. Secondly, the GUI model is defined by the developers using a language that offers limited GUI features, which makes the tool difficult to adopt in practice. Thirdly, it is currently not supported by any coding editor. For example, for validating these written models, features such as syntax highlighting, type checking, as well as some semantic feedback (e.g., constraint minimization) can be very useful. Furthermore, ActionGUI does not support n-ary associations and also requires careful FGAC permission design for symmetrical ends of binary associations. Finally, it is difficult to express object initialization constraints in ActionGUI.

Recently, Krstić et al. [6] extended MDS with the support for privacy (MDP) with a new model that captures *purpose-limitation* and *consent*. Given such a model, the generated enforcement code additionally enforces such classes of privacy policies. The approach is implemented in Python and besides the privacy policy enforcement it supports some of the ActionGUI's functionality.

[1] E.g., see the course on Security Engineering: `https://infsec.ethz.ch/education/as2022/seceng.html`

3 Objective

The objective of this semester project is to extend the implementation by Krstić et al. [6] to support all the features of ActionGUI and further improve it to mitigate its aforementioned drawbacks. Instead of relying on a high-level (GUI) model for specifying application behavior, the new tool should output code structured in such a way that developers can easily extend it with the desired behavior without affecting the enforcement. They can then also rely on any library that the target language supports. The tool should be able to compile models into a carefully design Python DSL that can alternatively be also used directly by programmers that want to avoid learning specific model syntax.

4 Tasks

1. Study the existing model-to-code transformation implemented in Python, MDS [2] and MDP [6].
2. Develop an ActionGUI-compatible model syntax and compile it into a Python DSL.
3. Create a VSCode client and a language server to highlight and validate the models, respectively.
4. (Optional) Extend the functionality of the language server to provide semantics-based feedback (e.g., constraint simplification, unsatisfiability...).
5. Add support for symmetrical constraints (by default).
6. Add explicit support for constraints on field initialization.
7. (Optional) Add support for complex model analysis, data model invariants, n-ary relations...
8. Develop multiple case studies (e.g., `Thoughts`, `MiniTwit` [1], `HIPAA`... etc) using the new approach, compare it to existing approaches, and assess its usability and performance.

5 Deliverables

The following deliverables are due at the end of the project:

Source code Complete development including easily reproducible evaluation.

Presentation At the end of the project, a 20 minute presentation must be given in an InfSec group seminar. It should give an overview and discuss the most important highlights of the work.

References

[1] MiniTwit: A tiny sample Flask app featuring a database and user accounts, November 2022. `https://github.com/Viasat/minitwit`.

[2] David Basin, Manuel Clavel, Marina Egea, Miguel Angel García de Dios, and Carolina Dania. A Model-Driven Methodology for Developing Secure Data-Management Applications. *IEEE Trans. Software Eng.*, 40(4):324–337, 2014.

[3] David Basin, Jürgen Doser, and Torsten Lodderstedt. Model driven security: from uml models to access control infrastructures. *ACM Transactions on Software Engineering and Methodology*, 15(1):39–91, January 2006.

[4] CMS. Gdpr enforcement tracker. `https://www.enforcementtracker.com/`, 2024.

[5] European Parliament and Council. Regulation (EU) 2016/679, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`.

[6] Srdan Krstic, Hoang Nguyen, and David A. Basin. Model-driven privacy. *Proc. Priv. Enhancing Technol.*, 2024(1):314–329, 2024.

[7] Open Worldwide Application Security Project. OWASP Top 10. `https://owasp.org/Top10/2025/`, 2025.

[8] Ravi S Sandhu and Pierangela Samarati. Access control: principle and practice. *IEEE communications magazine*, 32(9):40–48, 1994.